Writing for code review and software

Last modified: 2026-10-15 22:08:24 (PDT)

Software projects depend on short documents:

- pull requests
- issues
- commit messages
- READMEs
- code comments
- release notes

Readers of these documents often arrive without context on the change, so the rules in the other chapters apply to them too. This chapter lists the general rules that matter most here and adds a few that belong to software.

Apply the general rules

- Write in the active voice and name the actor before the action.
- Limit complex sentence structures: write one main idea per sentence and avoid piling several subordinate clauses into one.
- Cut filler and prefer plain words (Word choice).
- Scan for clichés before you post non-trivial prose. Skip the scan for code, terse status lines, and short replies, which are too short to contain a cluster of tells.
- Put lists in bullet points.

Say what changed and why

- Open a pull request description with the behavior a reader would see before and after the change.
- Explain the reason for the change in the commit message, not only the content of the diff.
- State what you checked and what you did not.
- Link the issue, the data, or the document that motivates the change.

Update plotting code.

Label the y-axis in original units, so the figure no longer shows log-odds.

Comment code sparingly

- Write a comment that says why the code does what it does, and let the code say what it does.
- Keep each comment as short as it can be while still giving the reason.
- Update or delete a comment when the code changes.

Write release notes for the user

- Describe each change by its effect on the user, not by the file that changed.
- Put breaking changes first, because the user must act on them.

> **Source**
>
> This chapter's scope, plain prose in software project text, follows the Writing chapter of the UCD-SERG lab manual.